University of Geneva
Department of Informatics

Methods and Heuristics for Learning and Optimization

Simulated Annealing and Parallel Tempering for the Traveling Salesman Problem

14X013

Michel Jean Joseph Donnet

October 24, 2025

Table of Contents

1 Introduction

(0.25)
be specific on

- search space
- neighborhood
- fitness
- TSP
- greedy
- SA

Human have been always interested in optimization problem to avoid a loss of energy or money or something else. In our digital world, the optimization play an important role in most domains. For instance, a travel compagny will try to maximise its customer base by proposing the customer to visit a maximum number of cities with minimal cost for him. Depending on the optimization of this problem, the compagny will have more customers and will develop more than another compagny in the same domain. To deal with theses optimization issues, heuristics and metaheuristics algorithm are created, using different approaches and modelizations of the real problem to propose an optimization. For instance, in case of the travel compagny, the problem can be modelized by finding a route that minimizes costs and passes through all the cities before returning to the agency. Each city then represents a node in a graph, and each city is connected to the others by an edge weighted by the price of travel between those two cities. This modelization of the problem is called Traveling Salesman Problem (TSP) and is known to be a NP-hard (Nondeterministic Polynomial-time hard) problem, that means that we can't find a solution in polynomial time. To find a solution that may not necessarily be optimal, we use heuristics and metaheuristics algorithm.

In this work, we will try to find a solution to the TSP problem by a metaheuristic method called Simulated Annealing method.

2 Methodology

(1.5)

First, we will explain some terms and definitions used in the rest of this report.

2.1 Modelization and definitions

The TSP problem can be modelized by N cities to visit, numeroted from 0 to $N-1$, with a cost matrix W that gives at the position i, j the cost of the travel between city i and city j. Each city is connected to each others, so the graph is fully connected. In this work, we assume that the cost of the travel between city i and j is the same in both directions. So the cost matrix W is symetric because $W_{ij} = W_{ji} \; \forall \; i, j \; \in \{0, \cdots, N-1\}$, which implies that $W = W^T$. Each solution can be written as a list of N cities in the order of visit. When the last city of the list is reached, the customers returns to the agency in the first city of the list. For instance, for $N = 3$ cities, a solution can be $[3, 1, 2]$ that means the customer will first visit city 3, then city 1 and finaly city 2 and then return to agency in city 3.

2.1.1 Search space

The search space gives the set of all possibles solutions for a given problem. In specific case of TSP problem, the search space will be the set of all the cyclic paths that go through all the cities.

As the paths are cyclic, it means that for instance path $[1, 2, 3]$ is the same that $[3, 1, 2]$. For N cities, for a given path, it's possible to perform N cyclic rotations before returning on the given path.

As we have assumed that the cost between city i and j is the same in both directions, it means that for instance path $[1, 2, 3]$ is the same that $[3, 2, 1]$. So for N cities, for a given path with its set of cyclic rotations, if we perform an inversion, we will obtain an inversed path with its corresponding set of cyclic rotations.

So a path with a given cost can have $2N$ variants.

So for N cities, the search space will be of size S given by the formula 1.

$$S = \frac{\text{Number of possible paths}}{\text{Number of variant per path}} = \frac{N!}{2N} = \frac{(N-1)!}{2} \tag{1}$$

So with a great N, the search space becomes astronomic.

Example: $N = 15 \implies S = \frac{(15-1)!}{2} = 4.3 \times 10^{10}$

2.1.2 Neighborhood

The neighborhood returns a set of solutions closest to the given solution by slightly modifying the given solution. It depens on the modelization of the problem. In our case,

each solution is a list of cities in order of visit.

We define the neighborhood like a swap of 2 elements of the current solution that gives another solution. The neighbor solution obtained must differs from the current solution and also from all cyclic rotations and inversions of the current solution.

So for a solution given with N cities, the size of the neighborhood is borned by the number of unique permutations of 2 elements in a list, like the formula 2 show us. The born is the number of permutations of 2 elements in a list because all the permutations will not be taken since certain permutations can result in a cyclic or inverse rotation of the solution.

$$\text{Number of neighbors } \leq \binom{N}{2} = \frac{N!}{2!(N-2)!} = \frac{N(N-1)}{2} \tag{2}$$

2.1.3 Fitness

The fitness, or cost function, returns the total cost of the travel around all the cities, according to the weight matrix W. As explained above, the cost between city i and city j is given by the element i, j of the cost matrix W which is symetric.

So we can define a fitness function as the sum of the cost of all travel between cities in a given solution x, as shown in the formula 3.

$$f(x) = W_{x_N x_1} + \sum_{i=2}^{N} W_{x_{i-1} x_i} \tag{3}$$

Note that the term $W_{x_N x_1}$ in the formula 3 design the cost of returning to the agency.

2.2 Heuristic vs metaheuristic

2.2.1 Heuristic

An heuristic approach to an optimization problem will try to find a good solution in polynomial time. It's a deterministic approach, which means that with same input, the method will give always the same output: nothing is given to the hasard. However, the heuristic approach is specific to each problem. Due to its deterministic nature, the algorithm will tend to be blocked by local optimal solutions in case of complex problems.

For instance, an heuristic algorithm that find the minimum of a function will pretty work on simple function like squared functions. However, for a complex function that admits a lot of local minimum, the heuristic algorithm will be blocked in a local minimum because for the algorithm, the minimum is found.

A greedy algorithm is a heuristic algorithm which make the best choice at each step. In this way, we hope to reach the global optimum.

2.2.2 Metaheuristic

A metaheuristic approach will try to find a good solution for a given problem by exploring the search space 2.1.1 of the problem. It's a stochastic approach, that means that probabilities are integrated to choice a solution for next step, that allow to don't take always the optimum solution. The stopping criteria is not the found of a good solution, but it is defined by the user through the guided parameters. The guided parameters are customisable by user and indicate in most case how, where and how long the metaheuristic algorithm will search.

Since the algorithm does not stop when it finds a solution and have some non-deterministic mechanism to choose next solution, theses prevent it from being blocked by a local solution, compared to heuristic algorithms.

So metaheuristic algorithms are good for complex problems. However, for simple problems, heuristic algorithm are more efficient because they will not make a big exploration of the search space like a metaheuristic algorithm can make, especially if the guided parameters are not properly set.

Metaheuristic algorithms are relatively simple to implement and don't depend on given problem. The challenge in metaheuristic algorithms result in the choice of guided parameters, which balance exploring the search space and exploiting the regions already visited.

2.2.3 Comparison

The table 1 describe the differences between heuristic and metaheuristic algorithms.

Table 1: Comparison between heuristic and metaheuristic algorithms

Feature	Heuristic	Metaheuristic
Method type	deterministic	often stochastic
Approach	problem-based	general
Implementation	often simple	often simple
Speed	fast	slower
Problem type	well-defined, structured problems	complex or poorly understood problems
Stopping criteria	fixed (e.g. solution found)	user-defined (e.g. time, iterations, convergence)
Local optima	often stuck	can escape
Use case	when problem structure is known	when heuristics are ineffective

In our case, we will try to implement both heuristic and metaheuristic algorithms: a greedy algorithm for heuristic and an algorithm that follows the simulated annealing

method for metaheuristic. Then, we will compare obtained results.

2.3 Simulated Annealing

The simulated annealing is inspired on the physical annealing process in metallurgy.

2.3.1 Physical inspiration

In metallurgy, to improve quality and capabilities of a metal, the metal is heated to high temperature and then cooled slowly. This is based on the fact that every physical system tends toward a state of equilibrium where energy is minimal. So when the metal is heated, the atoms of metal will have enough energy to move inside the metal. Then, they will position themselves in such a way as to reduce energy consumption. Their new positions will be more ordered than before heating the metal. This will create a better structured metal, which will improve quality and capabilities of the metal, like making it less brittle, more flexible or something else.

2.3.2 Algorithm principe

For the simulated annealing method in algorithmics, it will be the same process.

At the begining of the algorithm, the temperature will be very high, allowing a great exploration of the search space. In fact, with a great temperature, the system will have enough energy to make big jumps accross the search space.

Then, the temperature will decrease, which will force the exploitation. It's due to the fact that with a smaller temperature, the system is no longer able to achieve big jumps accross the search space.

Finaly, the algorithm will run until a state of equilibrium is reached, with a temperature more and more cold. The state of equilibrium is in general user defined. In our case, the state of equilibrium is reached when no improvement of fitness is made during last 3 changes of temperature.

2.3.3 Algorithm structure

The algorithm can be represented by the Figure 5 in appendix 7.

It basically works like this:

- initial configuration and temperature. In general, the initial configuration is choosen randomly. The temperature is a guided parameter. In our case, we take 100 random configurations in the entire search space and compute the mean of the absolute value of the fitness difference ΔE of theses points with the initial configuration. Then, the initial temperature T_0 is defined such as $e^{\frac{-\langle \Delta E \rangle}{T_0}} = 0.5$ with 0.5 the acceptance rate of a given configuration of the search space. The global search allows to give a global view of the variance of fitness inside the search space.

- Update configuration according to the metropolis rule given by the formula 4.

$$P = \begin{cases} 1 & \text{if } \Delta E < 0 \\ e^{\frac{-\Delta E}{T}} & \text{otherwise} \end{cases} \tag{4}$$

ΔE is the fitness difference between the new configuration and the current configuration. So the probability P to choose the given element is always 1 if this element gives an improvement to the current fitness, and is of $e^{\frac{-\Delta E}{T}}$ otherwise, as described in the formula 4. In other words, it means that if the new configuration is an improvement of the current configuration, it will be directly accepted. And if the new configuration is not an improvement of the current configuration, it means that the difference is positive and the new configuration will be accepted with a probability that decrease exponentialy as the difference grows, as shown on Figure 1.[1]

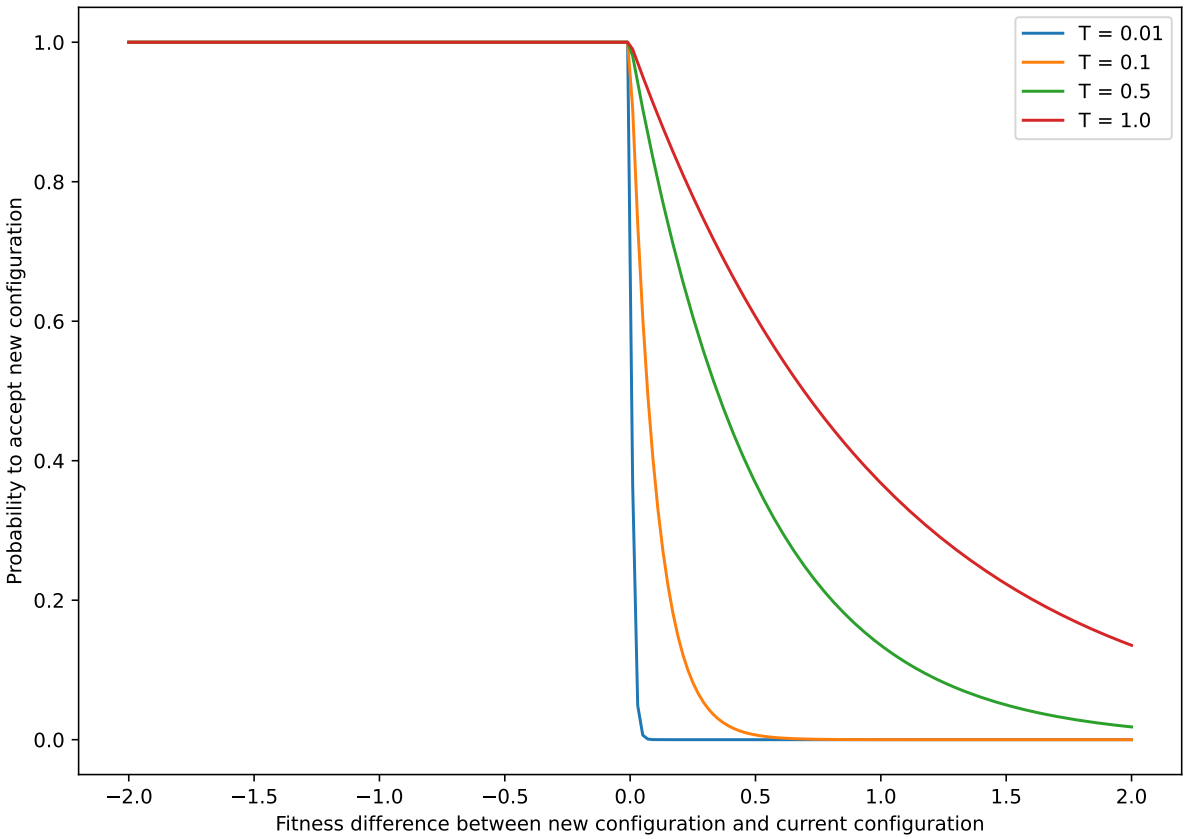


Figure 1: Probability to choose configuration

- If an equilibrium state is reached, update temperature according to the temperature schedule (a guided parameter). In general, the temperature variation is defined as a geometric sequence, that is given in our case by: $T_{k+1} = 0.9 \times T_k$. The equilibrium state is reached when enough configurations have been accepted of when enough

[1]For details about impact of temperature on the search, see section 2.3.3

configurations have been visited (configurations accepted + not accepted) with the given temperature. In our case, to reach equilibrium state, $12N$ configurations must be accepted or $100N$ configurations must be visited with N the number of cities.
- Iterate until the halting condition is reached. The halting condition is in general given by no improvement of the fitness during last X temperature changes (in our case, $X = 3$).
- Return the best configuration that gives the best fitness.

As it is easily possible to see, the method depends on a few guided parameters, like the initial temperature and the update of the temperature… This represents a challenge to set them correctly, and the quality of the result depends on this guided parameters. That's why the main difficulty of the simulated annealing method, like other metaheuristics methods, is, among other things, to correctly set the guided parameters.

Impact of the temperature The temperature plays a crutial role in the simulated annealing method. We speak about high and low temperature, but what is the impact of the temperature on the choice of the next configuration ? To answer, we will see 2 cases.

1. *Low temperature.* When the temperature is low, as we can see on the Figure 1, the probability to accept an element that is worse than the current configuration is very low, so we accept only configurations that are at least as good as the current configuration. The current configuration is therefore exploited since we try to find a better configuration around the given configuration.
2. *High temperature.* When the temperature is high, as the Figure 1 show us, the probability to accept worse configurations is high. So we try to explore further the search space to find better configuration by accepting more configurations.

2.3.4 Algorithm variant: the Parallel Tempering (PT)

There exists different variants of the simulated annealing algorithm, such as parallel tempering.

This variant is based on the same principe, but works quite differently, with different guided parameters. The parallel tempering algorithm can be represented by the diagram 6 in appendix 7.

The principle is to explore space in several locations. To achieve this goal, M replicas of the simulated annealing method are made. A list of M temperature is set such as $T_1 < \cdots < T_M$. This list describe the fixed temperature T_i for the simulated annealing replica i. A swap between adjacent replicas is made at a given frequency.

In more details, the parallel tempering works as follows:

- Initial configuration: M random configurations are choosen. In general, $M = \sqrt{N}$.
- Initial temperature: a list of temperature is defined with $T_1 < T_2 < \cdots < T_M$.

- For each current configuration i, choose a new configuration according to the Metropolis rule 4 with T_i as temperature.
- Each n iterations, swap configurations according to the fitness of theses configurations. The swaps are only attempted between configurations with adjacent temperatures. So for instance, configuration i cannot be swapped with configuration j if there exists a configuration k such as $T_i < T_k < T_j$. Δ_{ij} is then defined as $(fitness_j - fitness_i)(\frac{1}{T_j} - \frac{1}{T_i})$ with $i < j$. The Metropolis rule is then applied to Δ_{ij} to decide if the swap is accepted or not, as described in the formula 5.

$$P = \begin{cases} 1 & \text{if } \Delta_{ij} < 0 \\ e^{-\Delta_{ij}} & \text{otherwise} \end{cases} \tag{5}$$

  In this way, as explained in section [-sec-temp], when we have a good configuration, we exploit it by decreasing the temperature, and when we have a bad configuration, we want to explore more the search space to find better configurations by increasing the temperature.
- Iterate until halting condition is reached. In our case, the halting condition is the number of iterations.

So this algorithm can be an improvement of the simulated annealing system depending on the problem, and a parallelization of the method, as its name suggests, is possible to achieve, allowing more exploration of the search space. However, like for each metaheuristics, the guided parameters such as M, the list of temperatures $T_1 < \cdots < T_M$ and the maximum number of iterations must be properly defined to have a method that works quite well.

3 Implementation

The implementations of both simulated annealing and parallel tempering are based on the diagrams 5 and 6, included in the appendix 7. However, some pseudo-code are provided in sections below.

3.1 Requirements and usage

The code is written in python.

The figures are made with the python library Plotly that allows more control on graphs than `matplotlib.pyplot` library.

The requirements are:

- `python 3`
- `numpy`: library used to work with arrays
- `argparse`: library used to manage command line arguments
- `plotly`: python library used to plot the graphics. Documentation can be found on official website: `https://plotly.com/python/`
- `os`: library that allows to interact with operating system
- `time`: library used to capture execution time of algorithms

The code works in the command line, according to the following documentation:

```
usage: tp2.py [-h] [-b BENCHMARK] [-r RANDOM] [-d DATAFILE] [-t THRESHOLD]

TP3 - Traveling Salesman Problem

options:
  -h, --help            show this help message and exit
  -b, --benchmark BENCHMARK
                        Benchmark problem of N cities (trivial solution
                        for humans !)
  -r, --random RANDOM   Random problem of N cities
  -d, --datafile DATAFILE
                        Path to data file for problem with given data
  -t, --threshold THRESHOLD
                        Acceptance rate of algorithms. Default is 50%
```

The output of the program is some graphics and tables of TSP problem solutions for greedy, simulated annealing and parallel temperin methods.

For instance, to generate benchmark graphs for 30 cities, use the following command:

```
python tp2.py -b 30 -t 0.05
```

And to generate graphs for `cities.dat` specific problem, use the following command:

```
python tp2.py -d /path/to/cities.dat
```

3.2 Pseudo-codes

We are looking for path that goes across N cities. The set of cities is given, and the cost of a travel between 2 cities is given by distance between theses 2 cities.

3.2.1 Greedy

- initial step: choose randomly an initial city and put it in the path
- loop until each city is in the path:
  - compute distance between given city and the rest of the cities
  - choose city with smallest distance
  - add city to the path
- return the path

3.2.2 Simulated annealing

- initial step:
  - choose randomly a path. If `enhanced_mode` is activate, choose a path according to an optimization method (combination of greedy and simulated annealing)
  - choose initial temperature:
    - compute average of fitness change $\langle|\Delta E|\rangle$ for 100 random paths
    - take initial temperature T_0 such as $e^{-\frac{\langle|\Delta E|\rangle}{T_0}} = threshold$ with $threshold$ the acceptance rate of a path in the search space.
- loop until no improvement of fitness is made during last 3 temperature changes
  - choose randomly a new path from current path (by swapping 2 cities)
  - compute probability of acceptance according to Metropolis rule 4
  - if accepted
    - update path
    - if updated path is better than best path found so far
      - update best path
      - indicate that an improvement of fitness is made

  - if number of paths attempted is greater than $100N$ or number of paths accepted is greater than $12N$
    * update temperature according to `T_schedule` given: $T_{k+1} = \text{T\_schedule} \times T_k$

- return best path with best fitness

3.2.3 Parallel Tempering

The number of replicas of simulated annealing is given by M that is generaly defined like this: $M = \sqrt{N}$

1. initial step

choose randomly M paths. If `enhanced_mode` is activate, choose M paths according to an optimization method.
choose initial temperature:

- compute average of fitness change $\langle |\Delta E| \rangle$ for 100 random paths
- take initial temperature T_0 such as $e^{-\frac{\langle |\Delta E| \rangle}{T_0}} = threshold$ with $threshold$ the acceptance rate of a path in the search space.
- Get temperature list T multiplying T_0 by a sequence of factors that follow a `linear`, `quadratic`, `exponential` or `logarithmic` progression. For instance, for linear progression, we have $T = \frac{1}{N}T_0 < \frac{2}{N}T_0 < \cdots < T_0$

2. loop until max number of iterations is reached

for each path in paths

- choose randomly a new path (by swapping 2 cities)
- compute probability of acceptance according to Metropolis rule 4
- if accepted
  - update path
  - if updated path is better than best path found so far
    * update best path

each `swap_frequency` iteration

- swap adjacent replicas according to their fitness and Metropolis rule 5. For instance, if $fitness_1 < fitness_2$, swap replicas 1 and 2.

3. return best path with best fitness

4 Results

To compute the results, each method is runned 10 times.

The figure with the cities shows the best path during the 10 runs of the algorithm.

The table give always the average values obtained.

4.1 Benchmark problem

Benchmark: greedy for 30 cities

Attribute	Value
Best fitness	6.2717
Execution time (s)	0.0016

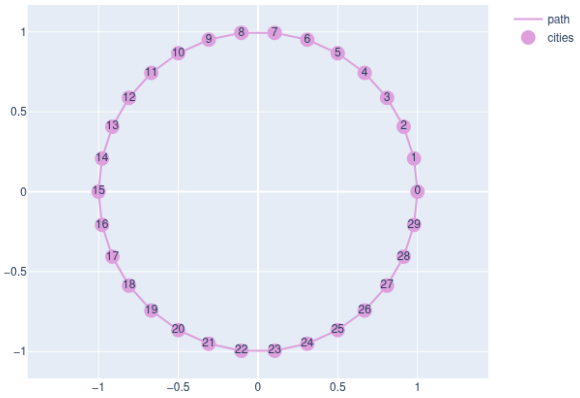


Figure 2: Benchmark problem of 30 cities for the greedy algorithm

Benchmark: simulated annealing for 30 cities

Attribute	Value
Best fitness	10.4294
Number iteration	42691.4
Initial temperature	1.1193
Enhanced mode	false
Initial acceptance rate	5%
Temperature schedule	$T_{k+1} = 0.9\ T_k$
Execution time (s)	4.2736

Attribute	Greedy	PT
Execution time ratio	0.0004	1.1898
Fitness ratio	0.6013	1.3116

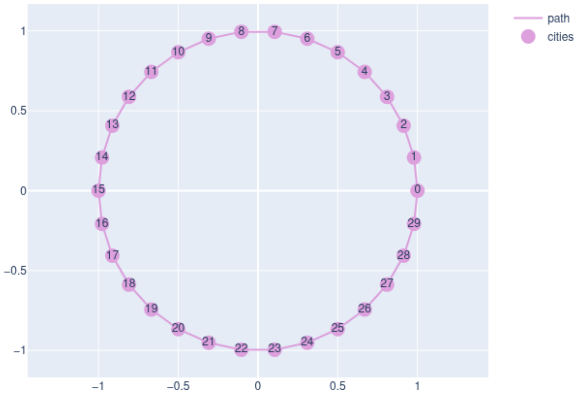


Figure 3: Benchmark problem of 30 cities for the simulated annealing algorithm

Benchmark: parallel tempering for 30 cities

Attribute	Value
Best fitness	13.679
Number iteration	5294.4
Initial temperature	1.3525
Temperature schedule type	exp
Enhanced mode	false
Initial acceptance rate	5%
Execution time (s)	5.0847

Attribute	Greedy	SA
Execution time ratio	0.0003	0.8405
Fitness ratio	0.4585	0.7624

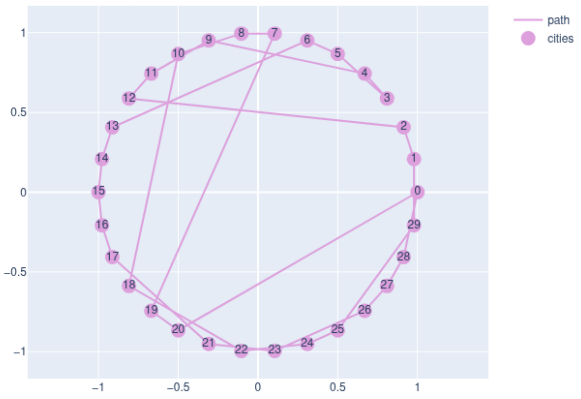
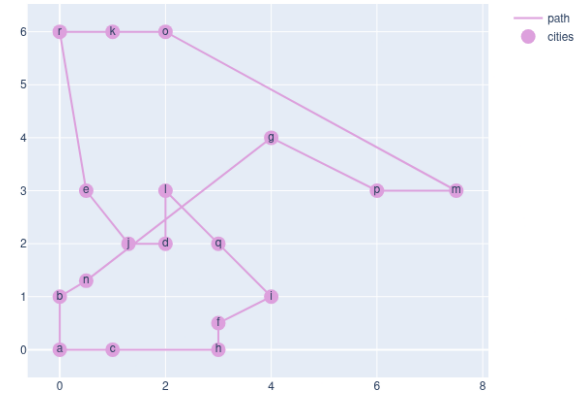


Figure 4: Benchmark problem of 30 cities for the parallel tempering algorithm

4.2 Cities problem

cities.dat: greedy for 18 cities

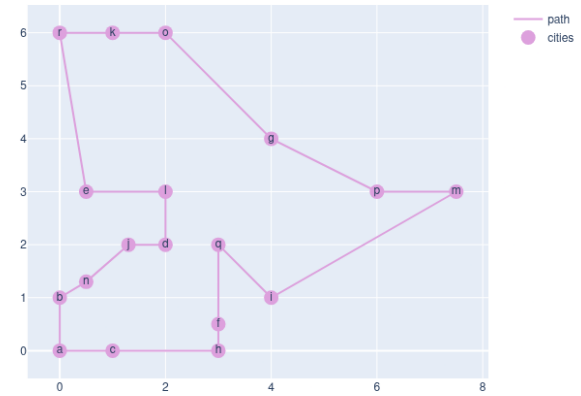
Attribute	Value
Best fitness	33.6173
Execution time (s)	0.0007



cities.dat: simulated annealing for 18 cities

Attribute	Value
Best fitness	31.7924
Number iteration	12702.8
Initial temperature	2.1198
Enhanced mode	false
Initial acceptance rate	5%
Temperature schedule	$T_{k+1} = 0.95\ T_k$
Execution time (s)	1.2652

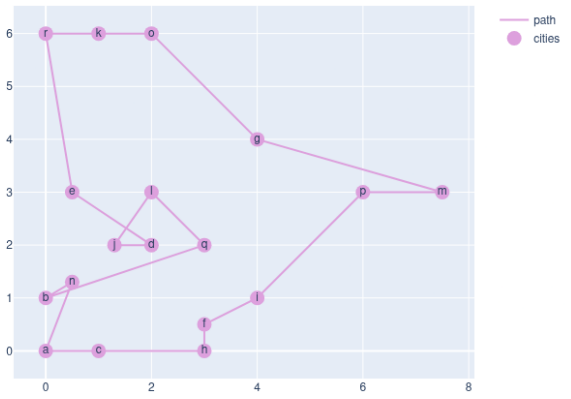
Attribute	Greedy	PT
Execution time ratio	0.0006	3.1601
Fitness ratio	1.0574	0.9273



cities.dat: parallel tempering for 18 cities

Attribute	Value
Best fitness	29.482
Number iteration	5082.5
Initial temperature	1.8774
Temperature schedule type	quad
Enhanced mode	false
Initial acceptance rate	5%
Execution time (s)	3.9981

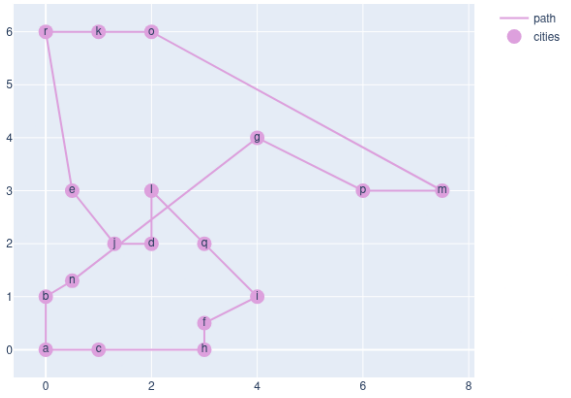
Attribute	Greedy	SA
Execution time ratio	0.0002	0.3164
Fitness ratio	1.1403	1.0784



4.3 Cities2 problem

cities.dat: greedy for 18 cities

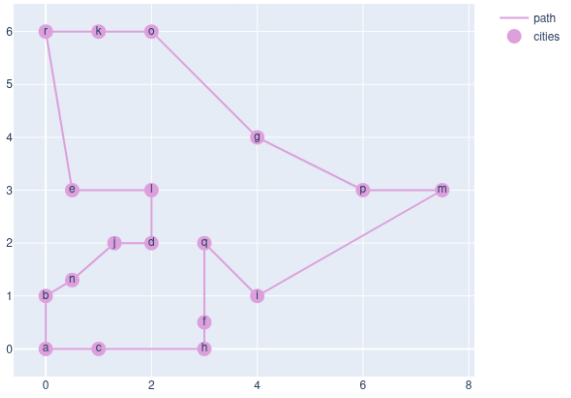
Attribute	Value
Best fitness	33.6173
Execution time (s)	0.0007



cities.dat: simulated annealing for 18 cities

Attribute	Value
Best fitness	31.7924
Number iteration	12702.8
Initial temperature	2.1198
Enhanced mode	false
Initial acceptance rate	5%
Temperature schedule	$T_{k+1} = 0.95\ T_k$
Execution time (s)	1.2652

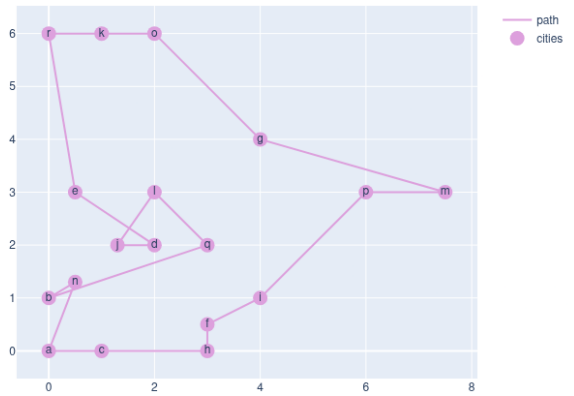
Attribute	Greedy	PT
Execution time ratio	0.0006	3.1601
Fitness ratio	1.0574	0.9273



cities.dat: parallel tempering for 18 cities

Attribute	Value
Best fitness	29.482
Number iteration	5082.5
Initial temperature	1.8774
Temperature schedule type	quad
Enhanced mode	false
Initial acceptance rate	5%
Execution time (s)	3.9981

Attribute	Greedy	SA
Execution time ratio	0.0002	0.3164
Fitness ratio	1.1403	1.0784



(1)

- circle
- cities
- cities2
- 20
- 60
- 80
- 100

Temperature change execution time
Just show plots and results, not discussion…

5 Discussion

(2)

In this section, we will discuss the results obtained and displayed in the previous section.

5.1 Benchmark problem

5.1.1 Quality

The benchmark problem defined is N cities distributed equally around a circle of radius 1. In this way, it's easy to know the optimal fitness: it's the path that create the circle and that have a fitness of $2\pi \approx 6.2$.

When we look at the results, we can see that the greedy algorithm find the optimal solution for this benchmark problem, as shown on Figure 2. It's the expected result, so the algorithm is well implemented.

However, we can constate that on the 10 executions, the simulated annealing and parallel tempering algorithms don't find always the optimal result because the best fitness mean exeeds that of the greedy best fitness mean, as illustrated in Figure 3 and Figure 4.

5.1.2 Execution time

If we look at the problem from the perspective of computation time, the greedy algorithm is

- greedy vs simulated annealing (SA)
- abrupt temperature change
- diversification
- intensification

6 Conclusion

(0.25)

7 Appendix

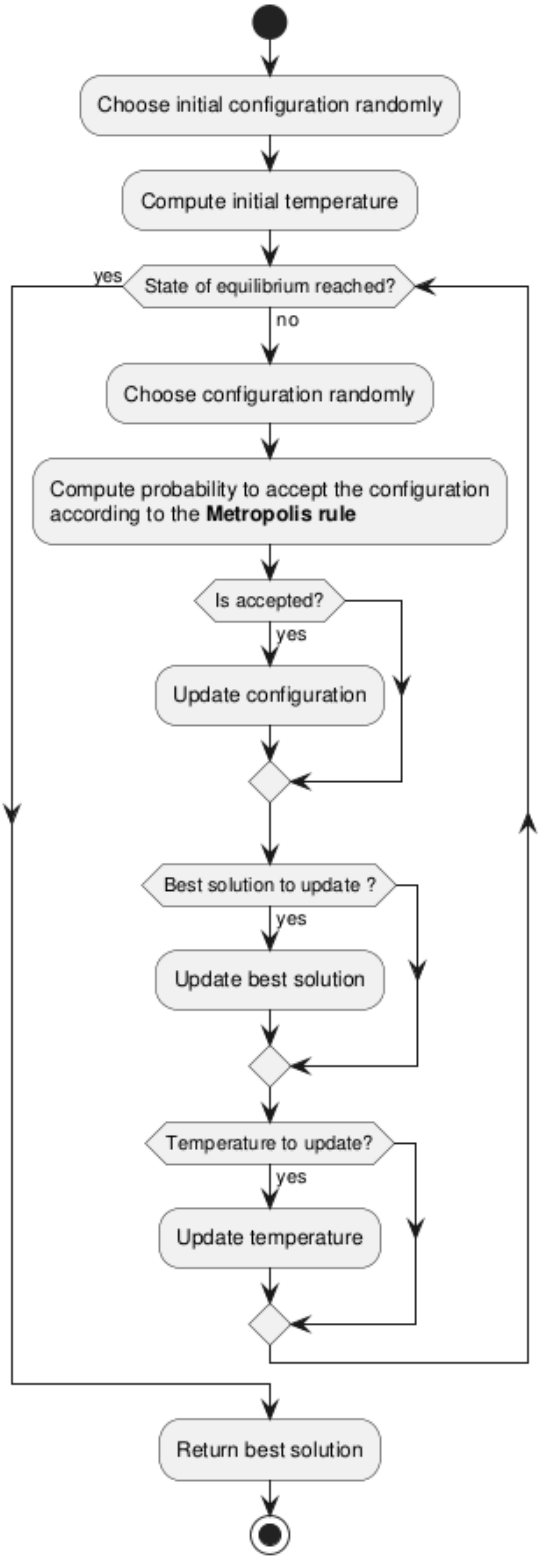


Figure 5: Simulated Annealing Diagram

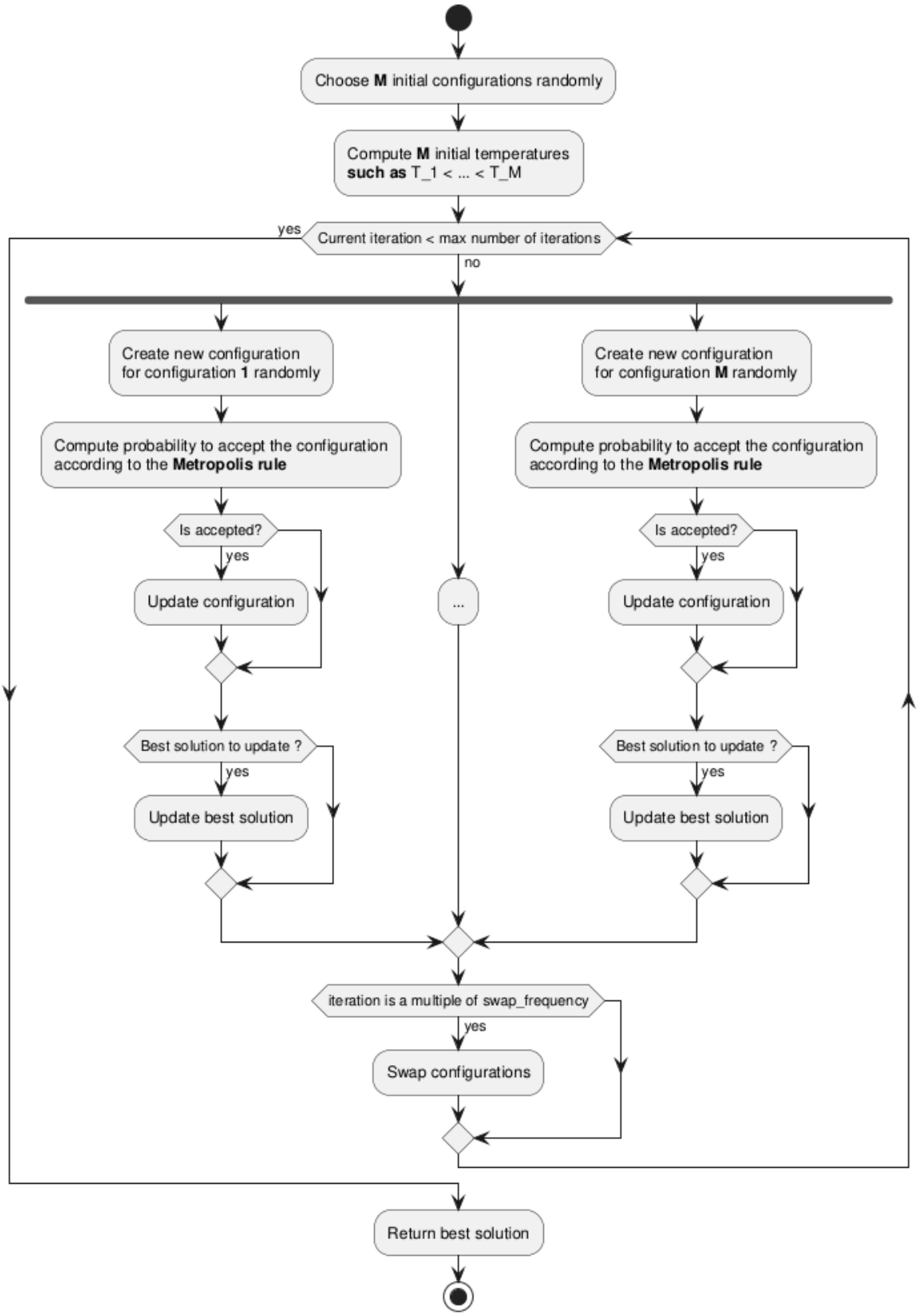


Figure 6: Parallel Tempering Diagram